

Conditions and Loops

PTP Program



Training Guidelines

Persyaratan dan Ekspektasi untuk Peserta Pelatihan:

1. **Prerequisite:** Peserta diharapkan sudah memiliki pengetahuan dasar tentang **tipe data di python**.
2. **Active Participation:** Peserta harus terlibat aktif dengan materi pelatihan, mencatat poin-poin penting dan penjelasan yang diberikan oleh instruktur.
3. **Clear Instruction:** Instruktur bertanggung jawab untuk menyampaikan materi pelatihan dengan cara yang jelas dan ringkas, memastikan semua peserta memahami topik yang dibahas.
4. **Hands-on Practice:** Sebagian besar pelatihan akan melibatkan latihan praktis yang dilakukan menggunakan **Google Colab (Jupyter Notebook)**. Anda dapat mengakses Google Colab yang tersedia di slide terakhir.

Contents

- Basic Conditional IF
- Nested and Ternary Operator
- Intro to Loop
- While Loop
- For Loop
- Exercise

Basic Conditional IF

IF Statement

Kita akan mulai dengan melihat jenis pernyataan if yang paling dasar. Dalam bentuk yang paling sederhana, pernyataan ini terlihat seperti ini:

```
if <expr>:  
    <statement>
```

Dalam bentuk di atas:

- **<expr>** adalah ekspresi yang dievaluasi dalam konteks Boolean.
- **<statement>** adalah pernyataan Python yang valid, yang harus **terindeks** (indentasi).

Jika **<expr>** bernilai benar (evaluasi menghasilkan nilai yang "truthy"), maka **<statement>** akan dieksekusi. Jika **<expr>** bernilai salah, maka **<statement>** akan dilewati dan tidak dieksekusi.

Basic Conditional IF

Grouping Statement

Dalam program Python, pernyataan-pernyataan yang berurutan dan diindentasikan pada level yang sama dianggap sebagai bagian dari blok yang sama. Oleh karena itu, pernyataan **if** majemuk (compound if statement) dalam Python terlihat seperti ini:

```
if <expr>:  
    <statement>  
    <statement>  
  
... <statement>  
<following_statement>
```

Basic Conditional IF

Elif & Else Clause

Terkadang, Anda ingin mengevaluasi suatu kondisi dan mengambil satu jalur jika kondisi tersebut benar, tetapi menentukan jalur alternatif jika kondisi tersebut salah. Ini dapat dilakukan dengan menggunakan klausa **else**:

```
if <expr>:
    <statement(s)>
elif <expr>:
    <statement(s)>
elif <expr>:
    <statement(s)>

    ...

else:
    <statement(s)>
```

Penjelasan:

- Jika **<expr>** bernilai **True**, maka **<statement1>** dieksekusi dan **<statement2>** dilewati.
- Jika **<expr>** bernilai **False**, maka **<statement1>** dilewati dan **<statement2>** dieksekusi.
- Setelah itu, eksekusi program akan dilanjutkan setelah blok **else** (setelah **<statement2>**).

Kedua blok pernyataan tersebut (baik dalam **if** maupun **else**) ditentukan dengan indentasi yang tepat, sebagaimana dijelaskan sebelumnya.

Nested IF and Ternary Operator

Nested IF Construct

Terkadang, Anda mungkin ingin memeriksa kondisi lain setelah sebuah kondisi sebelumnya bernilai **True**. Dalam hal ini, Anda dapat menggunakan *nested if* (if bersarang), di mana Anda menempatkan sebuah konstruksi **if...elif...else** di dalam konstruksi **if...elif...else** lainnya. Berikut adalah contoh struktur *nested if*.

```
if <expr1>:  
    <statement>  
    <statement>  
    if <expr2>:  
        <statement>  
    elif <expr3>:  
        <statement>  
    else:  
        <statement>  
else:  
    <statement>
```

Nested IF and Ternary Operator

Ternary Operator

Conditional expressions (sering disebut sebagai "operator ternary") memiliki prioritas terendah di antara semua operasi Python. Operator ternary ini memungkinkan Anda untuk mengevaluasi sesuatu berdasarkan kondisi yang bernilai **True** atau **False**. Ini memungkinkan Anda untuk menulis sebuah **if-else** dalam satu baris, menggantikan penggunaan **if-else** yang memerlukan beberapa baris, dan membuat kode menjadi lebih ringkas.

```
<statement_if_true> if <expr> else <alternative_statement>
```


Introduction to Loop

Apa itu Loop?

Iterasi berarti mengeksekusi blok kode yang sama berulang kali, mungkin banyak kali. Struktur pemrograman yang menerapkan iterasi disebut **loop** (perulangan). Dalam pemrograman, ada dua jenis iterasi, yaitu **iterasi tak tentu** dan **iterasi pasti**:

1. **Iterasi Tak Tentu (indefinite)**
Pada iterasi tak tentu, jumlah berapa kali loop dieksekusi **tidak ditentukan secara eksplisit sebelumnya**. Sebaliknya, blok yang ditentukan akan dieksekusi berulang kali selama kondisi tertentu **terpenuhi**.
2. **Iterasi Pasti (definite)**
Pada iterasi pasti, jumlah berapa kali blok yang ditentukan akan dieksekusi **ditentukan secara eksplisit** saat loop dimulai.

While Loops

Basic Syntax of While Loops

Format dari *while loop* yang sederhana ditunjukkan di bawah ini:

```
while <expr>:  
    <statement(s)>
```

<statement(s)> mewakili blok kode yang akan dieksekusi berulang kali, yang sering disebut sebagai *body* dari loop.

Ketika sebuah *while loop* dijalankan, pertama-tama <expr> dievaluasi dalam konteks Boolean. Jika hasil evaluasinya **True**, maka body dari loop akan dieksekusi. Setelah itu, <expr> diperiksa lagi. Jika masih **True**, body akan dieksekusi lagi. Proses ini terus berulang hingga <expr> bernilai **False**.

While Loops

Break & Continue Statement

Python menyediakan dua kata kunci yang dapat menghentikan iterasi loop secara prematur:

- **break**: Pernyataan ini langsung menghentikan seluruh loop. Eksekusi program dilanjutkan ke pernyataan pertama setelah body loop.
- **continue**: Pernyataan ini langsung menghentikan iterasi loop saat ini. Eksekusi melompat ke bagian atas loop, dan ekspresi pengontrol loop akan dievaluasi kembali untuk menentukan apakah loop akan dieksekusi lagi atau dihentikan.

While Loops

Break & Continue Statement

Contoh *break* statement:

```
n = 5
while n > 0:
    n -= 1
    if n == 2:
        break # Break Statement
    print(n)
print('Loop ended.')
```

Contoh *continue* statement:

```
n = 5
while n > 0:
    n -= 1
    if n == 2:
        continue
    print(n)
print('Loop ended.')
```

While Loops

Else Clause

Python memungkinkan penggunaan klausa `else` opsional di akhir loop `while`. Ini adalah fitur unik Python yang tidak ditemukan di banyak bahasa pemrograman lainnya. Sintaksisnya ditunjukkan di bawah ini:

```
while <expr>:  
    <statement(s)>  
else:  
    <additional_statement(s)>
```

Pernyataan `<additional_statement(s)>` yang ditentukan dalam klausa `else` akan dieksekusi ketika loop `while` berakhir, **hanya jika** loop berakhir secara normal (bukan karena pernyataan `break`).

Jika loop dihentikan oleh `break`, klausa `else` **tidak** akan dieksekusi.

While Loops

Nested While Loops

Secara umum, struktur kontrol di Python dapat saling bersarang satu sama lain. Misalnya, Anda dapat menempatkan `if` di dalam loop, atau menempatkan loop di dalam blok `if`, dan seterusnya.

```
a = ['foo', 'bar']
while len(a):
    print(a.pop(0))
    b = ['baz', 'qux']
    while len(b):
        print('>', b.pop(0))
```

For Loops

Basic Syntax of For Loop

Loop `for` di Python terlihat seperti ini:

```
for <var> in <iterable>:  
    <statement(s)>
```

Penjelasannya:

- `<iterable>` adalah koleksi objek—misalnya, sebuah list atau tuple.
- Pernyataan dalam tubuh loop (`<statement(s)>`) akan dieksekusi satu kali untuk setiap item dalam `<iterable>`.
- Variabel loop `<var>` akan mengambil nilai dari elemen berikutnya dalam `<iterable>` setiap kali loop dijalankan.

```
a = ['foo', 'bar', 'baz']  
for i in a:  
    print(i)
```

For Loops

range() Function

Python menyediakan opsi yang lebih baik — fungsi bawaan `range()`, yang mengembalikan iterable yang menghasilkan urutan bilangan bulat. Fungsi `range(<end>)` mengembalikan iterable yang menghasilkan bilangan bulat mulai dari 0, hingga (tetapi tidak termasuk) `<end>`.

```
x = range(5)
for n in x:
    print(n)
for n in x:
    print(n)
```


For Loops

Altering For Loop Behaviour

Pernyataan **break** digunakan untuk **menghentikan seluruh loop** dan melanjutkan eksekusi program ke pernyataan pertama yang berada setelah loop.

```
for i in ['foo', 'bar', 'baz', 'qux']:
    if 'b' in i:
        break
    print(i)
```

Pernyataan **continue** digunakan untuk **menghentikan iterasi saat ini** dan melanjutkan ke iterasi berikutnya dari loop tanpa mengeksekusi sisa kode dalam loop tersebut.

```
for i in ['foo', 'bar', 'baz', 'qux']:
    if 'b' in i:
        continue
    print(i)
```

For Loops

Ternary Operator of For Loops

Sama seperti pada kondisi IF, **For Loops** juga memiliki bentuk sintaks lain, yaitu **operator ternary**:

```
<statement> for <var> in <iterable_object>
```

Selain itu, **conditional IF** dan **for loops** dapat digabungkan bersama menggunakan **operator ternary**:

```
<statement> for <var> in <iterable_object> if <expr>
```

*Perlu dicatat bahwa jika Anda menggunakan kombinasi ini, Anda tidak bisa menggunakan **else** dan **elif**.*

Let's Go For

Exercise - 1

Tulis program Python untuk membuat pola berikut, menggunakan loop bersarang (nested loop).

Expected Output:

```
1
22
333
4444
55555
666666
7777777
88888888
999999999
```

Let's Go For

Exercise - 2

Tulis program Python untuk memeriksa keabsahan kata sandi yang dimasukkan oleh pengguna.

Validasi :

- Setidaknya 1 huruf antara [a-z] dan 1 huruf antara [A-Z].
- Setidaknya 1 angka antara [0-9].
- Setidaknya 1 karakter dari [\$#@].
- Panjang minimum 6 karakter.
- Panjang maksimum 16 karakter.



External References

Colab Link

[Visit Here](#)



Hacktiv8id



Hacktiv8id



Hacktiv8



Hacktiv8 Indonesia

